

Code Explanation

Delta Live Tables Pipeline Overview

This code defines a Delta Live Tables (DLT) pipeline that ingests customer and order data from CSV files, cleans and processes the data, and creates aggregated views for business insights. The pipeline is designed to handle incremental updates and implements Slowly Changing Dimension (SCD) Type 2 for customer data.

The pipeline consists of multiple tables, each with its own specific purpose:

- Bronze tables: raw data loaded from CSV files
- Silver tables: cleaned data with no nulls or duplicates
- Gold tables: business value tables with aggregated data and SCD Type 2 implementation

Step 1: Defining Schemas for Customer and Order Data

This step defines the schema for customer and order data using PySpark's `StructType` and `StructField`. The schema defines the structure of the data, including column names, data types, and nullability.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 2: Loading Raw Data into Bronze Tables

This step loads the raw customer and order data from CSV files into bronze tables using DLT's `@dlt.table` decorator. The data is read using PySpark's `spark.read.format("csv")` method, and the schema is applied to the data.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data loaded from CSV"
)
def customer_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .schema(customer_schema)
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customere
rdata")
    )
```

```

    )
@dlt.table(
    name="order_bronze",
    comment="Raw order data loaded from CSV"
)
def order_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .schema(order_schema)
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderda
ta")
    )

```

Step 3: Cleaning Data and Creating Silver Tables

This step cleans the data by removing nulls and duplicates, and creates silver tables using DLT's `@dlt.table` decorator. The data is filtered to remove nulls, and duplicates are removed using the `dropDuplicates` method.

```

@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with no nulls or duplicates"
)
def customer_silver():
    return (
        dlt.read("customer_bronze")
        .filter(
            (col("CustId").isNotNull()) &
            (col("Name").isNotNull()) &
            (col("EmailId").isNotNull()) &
            (col("Region").isNotNull())
        )
        .dropDuplicates(["CustId"])
    )

@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return (
        dlt.read("order_bronze")
        .filter(
            (col("OrderId").isNotNull()) &
            (col("ItemName").isNotNull()) &
            (col("PricePerUnit").isNotNull()) &
            (col("Qty").isNotNull()) &
            (col("Date").isNotNull()) &
            (col("CustId").isNotNull())
        )
        .dropDuplicates(["OrderId"])
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
    )

```

Step 4: Creating Gold Tables with SCD Type 2 Implementation

This step creates a gold table that combines customer and order data, and implements SCD Type 2 to track changes in customer data. The data is joined using an inner join, and the resulting data is processed to identify changes in customer data.

```
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table combining customer and order data",
    table_properties={
        "quality": "gold",
        "delta.enableChangeDataFeed": "true"
    }
)
@dlt.expect_or_fail("valid_custid", "CustId IS NOT NULL")
def ordersummary():
    # Get current state of the table if it exists
    try:
        existing_data = dlt.read("ordersummary")
        exists = True
    except:
        exists = False

    # Join customer and order data
    joined_data = (
        dlt.read("customer_silver")
        .join(
            dlt.read("order_silver"),
            "CustId",
            "inner"
        )
        .select(
            "CustId",
            "Name",
            "EmailId",
            "Region",
            "OrderId",
            "ItemName",
            "PricePerUnit",
            "Qty",
            "Date",
            "TotalAmount"
        )
    )

    # For initial load
    if not exists:
        return (
            joined_data
            .withColumn("IsActive", lit(True))
            .withColumn("StartDate", current_timestamp())
            .withColumn("EndDate", lit(None).cast(TimestampType()))
            .withColumn("ChangeHash", expr("md5(concat(Name, EmailId
, Region)))"))
        )
```

```

# For updates (SCD Type 2 implementation)
# Identify changes in customer data
customer_changes = (
    dlt.read("customer_silver")
    .withColumn("ChangeHash", expr("md5(concat(Name, EmailId, Re
gion)))")
    .alias("new")
    .join(
        existing_data.select("CustId", "ChangeHash").alias("old"
),
        col("new.CustId") == col("old.CustId"),
        "left"
    )
    .where(
        (col("new.ChangeHash") != col("old.ChangeHash")) |
        col("old.ChangeHash").isNull()
    )
    .select(
        col("new.CustId").alias("CustId"),
        col("new.ChangeHash").alias("NewHash")
    )
)

# Mark existing records as inactive
updated_existing = (
    existing_data
    .join(customer_changes, "CustId", "left")
    .withColumn(
        "IsActive",
        when(col("NewHash").isNotNull(), False).otherwise(col("I
sActive"))
    )
    .withColumn(
        "EndDate",
        when(col("NewHash").isNotNull(), current_timestamp()).ot
herwise(col("EndDate"))
    )
    .drop("NewHash")
)

# Create new active records
new_records = (
    joined_data
    .join(customer_changes, "CustId", "inner")
    .withColumn("IsActive", lit(True))
    .withColumn("StartDate", current_timestamp())
    .withColumn("EndDate", lit(None).cast(TimestampType()))
    .withColumn("ChangeHash", expr("md5(concat(Name, EmailId, Re
gion)))")
    .drop("NewHash")
)

# Union existing and new records
return updated_existing.unionByName(new_records, allowMissingCol
umns=True)

```

Step 5: Creating Aggregated Customer Spending Table

This step creates a gold table that aggregates customer spending by name and date.

```
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date"
)
def customeraggregatespend():
    return (
        dlt.read("ordersummary")
        .where("IsActive = true")
        .groupBy("Name", "Date")
        .agg(expr("sum(TotalAmount)").alias("TotalAmount"))
    )
```